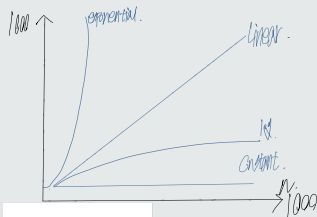


Workspace 'Week 1' in '6.006'

Page 1 (row 1, column 1)

input	constant	logarithmic	linear	log-linear	quadratic	polynomial	exponential
n	$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^k)$	$2^{\Theta(n)}$
1000	1	≈ 10	1000	$\approx 10,000$	1,000,000	1,000 ^k	$2^{1000} \approx 10^{301}$
Time	1 ns	10 ns	1 μ s	10 μ s	1 ms	10 ³⁰ s	10 ²⁸¹ millennia

这些时间复杂度
相差多少倍



Lecture 2: Data Structures

Interface is data structures

- What you want to do? How you do it?
- Representation
- What data on ops - How to store data.
- What the operation are important & what they mean
- Algorithms to support operations
- Problem
- Solution

2. main interface

- set

- sequence

[5, 2, 1, 7]

2. main data structures approaches

- Arrays

- pointer based

Sequence Interface

key: Word from model of computation.

Collection (natural): static array $A[i] = a_i$

- $O(1)$ per $\text{get_at}(\text{set_at})$

- $O(n)$ per build/rearrange

- memory: array of w -bit words

- "array": consecutive chunk of memory

$\Rightarrow \text{array}[i] = \text{memory}[\text{address}(\text{array}) + i]$

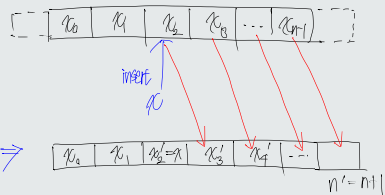
$\Rightarrow \text{array access is } O(1) \text{ time}$

Assume $w \geq n$

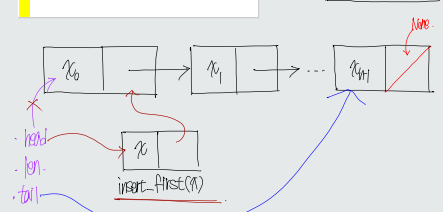
$\Rightarrow$ Memory allocation model: allocate array of size n in $O(n)$ time

space = $O(\text{time})$

Dynamic sequence interface:



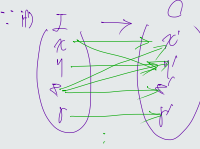
Linked List



Problem

$f: I \rightarrow O$

binary relation



$f(x) = x'$

$f(x) = y'$

这些映射关系
都是对的

Algorithm

$f: I \rightarrow O$

1st. problem state

2nd. input state

3rd. algorithm

4th. output state

5th. final state

6th. end state

7th. result

8th. final result

9th. final result

10th. final result

11th. final result

12th. final result

13th. final result

14th. final result

15th. final result

16th. final result

17th. final result

18th. final result

19th. final result

20th. final result

21th. final result

22th. final result

23th. final result

24th. final result

25th. final result

26th. final result

27th. final result

28th. final result

29th. final result

30th. final result

Page 3 (row 3, column 1)

∴ 证明 1 略 2 略 3 略 4 略

$\rightarrow$ let: initial
 initial initial initial
 initial

∴ Army 防衛隊

11) 细胞质核膜

· 继续朝朝朝朝朝

$$\therefore \text{原式} = \frac{1000 + i \cdot 1000}{1000 - 1000i} = \frac{1000(1+i)}{1000(1-i)} = \frac{1+i}{1-i}$$


: 雄 功 能 强 壮

Q622) 2018 (1044-111) 問題 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1

∴ 正则语言 ($\neq$ 正则列表)

∴ 正则语言 ($\neq$ 正则列表)

- 將 model 變為 data 的 model

• 有 Katak 与 强 格

· 附在 附錄 中 附錄 2 附錄 3

• 香港海港工程

∴ 世訓 New York 號

點 點 點 點 點 點 點 點
 能 能 能 能 能 能 能 能
 ⇒ 能 能 能 能 能 能 能 能
 點 點 點 點 點 點 點 點

∴ AIDY 和 强 磁 是 手 铐

[illegible]

== Python 呢

⇒ 此 处 初 始 值 为 0
: 0 1 2 3 4 5 6 7 8 9
11 12 13 14 15
for i in range(0, 16):
 if i % 2 == 1: 6(n) 为 奇 数 时

Amortized Analysis (amortized)

: 0 1 2 3 4 5 6 7 8 9
T(n) 为 0 1 2 3 4 5 6 7 8 9

Lecture 1: Introduction

The goal of this class is to teach you to **solve** computation problems, and to **communicate** that your solutions are **correct** and **efficient**.

Problem

- Binary relation from problem inputs to correct outputs
- Usually don't specify every correct output for all inputs (too many!)
- Provide a verifiable **predicate** (a property) that correct outputs must satisfy
- 6.006 studies problems on large general input spaces
- Not general: small input instance
 - **Example:** In this room, is there a pair of students with same birthday?
- General: arbitrarily large inputs
 - **Example:** Given any set of n students, is there a pair of students with same birthday?
 - If birthday is just one of 365, for $n > 365$, answer always true by pigeon-hole
 - Assume resolution of possible birthdays exceeds n (include year, time, etc.)

Algorithm

- Procedure mapping each input to a single output (deterministic)
- Algorithm **solves** a problem if it returns a correct output for every problem input
- **Example:** An algorithm to solve birthday matching
 - Maintain a **record** of names and birthdays (initially empty)
 - Interview each student in some order
 - * If birthday exists in record, return found pair!
 - * Else add name and birthday to record
 - Return None if last student interviewed without success

Correctness

- Programs/algorithms have fixed size, so how to prove correct?
- For small inputs, can use case analysis
- For arbitrarily large inputs, algorithm must be **recursive** or loop in some way
- Must use **induction** (why **recursion is such a key concept in computer science**)
- **Example:** Proof of correctness of birthday matching algorithm
 - Induct on k : the number of students in record
 - **Hypothesis:** if first k contain match, returns match before interviewing student $k + 1$
 - **Base case:** $k = 0$, first k contains no match
 - Assume for induction hypothesis holds for $k = k'$, and consider $k = k' + 1$
 - If first k' contains a match, already returned a match by induction
 - Else first k' do not have match, so if first $k' + 1$ has match, match contains $k' + 1$
 - Then algorithm checks directly whether birthday of student $k' + 1$ exists in first k' $\square$

Efficiency

- How fast does an algorithm produce a correct output?
 - Could measure time, but want performance to be machine independent
 - **Idea!** Count number of fixed-time operations algorithm takes to return
 - Expect to depend on size of input: larger input suggests longer time
 - Size of input is often called ' n ', but not always!
 - Efficient if returns in **polynomial time** with respect to input
 - Sometimes no efficient algorithm exists for a problem! (See L20)
- Asymptotic Notation: ignore constant factors and low order terms
 - Upper bounds (O), lower bounds (Ω), tight bounds (Θ) $\in, =, \text{is, order}$
 - Time estimate below based on one operation per cycle on a 1 GHz single-core machine
 - Particles in universe estimated $< 10^{100}$

input	constant	logarithmic	linear	log-linear	quadratic	polynomial	exponential
n	$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^c)$	$2^{\Theta(n^c)}$
1000	1	≈ 10	1000	$\approx 10,000$	1,000,000	1000^c	$2^{1000} \approx 10^{301}$
Time	1 ns	10 ns	1 μs	10 μs	1 ms	$10^{3c-9} s$	10^{281} millenia

Model of Computation

- Specification for what operations on the machine can be performed in $O(1)$ time
- Model in this class is called the **Word-RAM**
- **Machine word**: block of w bits (w is word size of a w -bit Word-RAM)
- **Memory**: Addressable sequence of machine words
- **Processor** supports many **constant time** operations on a $O(1)$ number of words (**integers**):
 - **integer** arithmetic: $(+, -, *, //, \%)$
 - **logical** operators: $(\&\&, ||, !, ==, <, >, <=, ==>)$
 - **(bitwise** arithmetic: $(\&, |, <<, >>, \dots)$
 - Given word a , can **read** word at address a , **write** word to address a
- Memory address must be able to access every place in memory
 - Requirement: $w \geq \#$ bits to represent largest memory address, i.e., $\log_2 n$
 - 32-bit words $\rightarrow$ max ~ 4 GB memory, 64-bit words $\rightarrow$ max ~ 16 exabytes of memory
- **Python** is a more complicated model of computation, implemented on a Word-RAM

Data Structure

- A **data structure** is a way to store non-constant data, that supports a set of operations
- A collection of operations is called an **interface**
 - **Sequence**: Extrinsic order to items (first, last, n th)
 - **Set**: Intrinsic order to items (queries based on item keys)
- Data structures may implement the same interface with different performance
- **Example: Static Array** - fixed width slots, fixed length, static sequence interface
 - `StaticArray(n)`: allocate static array of size n initialized to 0 in $\Theta(n)$ time
 - `StaticArray.get_at(i)`: return word stored at array index i in $\Theta(1)$ time
 - `StaticArray.set_at(i, x)`: write word x to array index i in $\Theta(1)$ time
- Stored word can hold the address of a larger object
- Like Python `tuple` plus `set_at(i, x)`, Python `list` is a **dynamic array** (see L02)

```

1 def birthday_match(students):
2     '''
3     Find a pair of students with the same birthday
4     Input: tuple of student (name, bday) tuples
5     Output: tuple of student names or None
6     '''
7     n = len(students)                # O(1)
8     record = StaticArray(n)          # O(n)
9     for k in range(n):               # n
10        (name1, bday1) = students[k]  # O(1)
11        # Return pair if bday1 in record
12        for i in range(k):           # k
13            (name2, bday2) = record.get_at(i) # O(1)
14            if bday1 == bday2:        # O(1)
15                return (name1, name2)  # O(1)
16        record.set_at(k, (name1, bday1)) # O(1)
17    return None                       # O(1)

```

Example: Running Time Analysis

- Two loops: outer $k \in \{0, \dots, n-1\}$, inner is $i \in \{0, \dots, k\}$
- Running time is $O(n) + \sum_{k=0}^{n-1} (O(1) + k \cdot O(1)) = O(n^2)$
- Quadratic in n is **polynomial**. Efficient? Use different data structure for record!

How to Solve an Algorithms Problem

1. Reduce to a problem you already know (use data structure or algorithm)

Search Problem (Data Structures)

Static Array (L01)
 Linked List (L02)
 Dynamic Array (L02)
 Sorted Array (L03)
 Direct-Access Array (L04)
 Hash Table (L04)
 Balanced Binary Tree (L06-L07)
 Binary Heap (L08)

Sort Algorithms

Insertion Sort (L03)
 Selection Sort (L03)
 Merge Sort (L03)
 Counting Sort (L05)
 Radix Sort (L05)
 AVL Sort (L07)
 Heap Sort (L08)

Shortest Path Algorithms

Breadth First Search (L09)
 DAG Relaxation (L11)
 Depth First Search (L10)
 Topological Sort (L10)
 Bellman-Ford (L12)
 Dijkstra (L13)
 Johnson (L14)
 Floyd-Warshall (L18)

2. Design your own (recursive) algorithm

- Brute Force
- Decrease and Conquer
- Divide and Conquer
- **Dynamic Programming** (L15-L19)
- Greedy / Incremental

MIT OpenCourseWare
<https://ocw.mit.edu>

6.006 Introduction to Algorithms
Spring 2020

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>